UNIVERSIDAD ORT
Uruguay

Facultad de Ingeniería
Bernard Wand-Polak
Cuareim 1451
11.100 Montevideo, Uruguay
Tel. 2902 15 05 Fax 2908 13 70
www.ort.edu.uy

EVALUACIÓN	**Obligatorio**	**GRUPO**		**FECHA**	
MATERIA	Infraestructura				
CARRERA	Licenciatura				
CONDICIONES	**- Puntaje máximo:** 35 puntos **- Puntaje mínimo:** 1 puntos **- Fecha de entrega:** 26/11/26 hasta las 21:00 horas en gestion.ort.edu.uy (max. 40Mb en formato zip, rar o pdf)				

Uso de material de apoyo y/o consulta

Inteligencia Artificial Generativa

- Seguir las pautas de los docentes: Se deben seguir las instrucciones específicas de los docentes sobre cómo utilizar la IA en cada curso.
- Citar correctamente las fuentes y usos de IA: Siempre que se utilice una herramienta de IA para generar contenido, se debe citar adecuadamente la fuente y la forma en que se utilizó.
- Se debe indicar:
    a. la /las herramientas utilizadas y
    b. el contexto de uso: por ejemplo, generación de ideas, redacción inicial, análisis de datos, corrección de estilo, etc.
- Todo contenido producido por IAG debe ser revisado y verificado; cualquier error presente será responsabilidad del estudiante.
- Ser responsables con el uso de la IA: Conocer los riesgos y desafíos, como la creación de "alucinaciones", los peligros para la privacidad, las cuestiones de propiedad intelectual, los sesgos inherentes y la producción de contenido falso.
- El uso de herramientas de Inteligencia Artificial Generativa (IAG) puede emplearse como apoyo en el proceso de aprendizaje. Sin embargo, no sustituye el razonamiento crítico ni la elaboración personal. El estudiante es responsable de que el trabajo presentado refleje su propia comprensión, análisis y proceso cognitivo sobre el tema.
- La defensa es obligatoria y eliminatoria.
- La defensa será presencial salvo en casos en que se habilite la defensa oral sincrónica en forma remota.

Defensa

Fecha de defensa:A acordar con el docente, previo al 09/07/26

La defensa es obligatoria y eliminatoria. El docente es quien definirá y comunicará la modalidad, y mecánica de defensa. La no presentación a la misma implica la pérdida de la totalidad de los puntos del Obligatorio.

IMPORTANTE:

1) Inscribirse
2) Formar grupos de hasta 3 personas del mismo dictado
3) Subir el trabajo a Gestión antes de la hora indicada (ver hoja al final del documento: "RECORDATORIO")

	Aquellos de ustedes que presenten alguna dificultad con su inscripción o tengan inconvenientes técnicos, por favor contactarse con la Coordinadora de cursos o Coordinación Adjunta **antes de las 20:00h** del día de la entrega, a través del mail: adjuntos_ei@ort.edu.uy.

Parte 1 - Bash

Sistema de Gestión de Flota "Don Drones"



Don Drones es una empresa de logística que utiliza drones autónomos para repartos. Cada dron pertenece a una **categoría** según su capacidad de carga:

Categoría	Descripción
Scout	Dron liviano, solo para reconocimiento y rutas cortas.
Cargo	Dron de carga estándar, hasta 5kg.
Heavy	Dron de carga pesada, hasta 20kg.
Cooler	Dron con compartimento refrigerado, para alimentos/medicamentos.
Racer	Dron de alta velocidad, para entregas urgentes.

Actualmente, el equipo de operaciones está utilizando el script `don_drones_manager.sh` para gestionar el inventario de drones disponibles en el depósito. Los datos se guardan en `flota.csv` con las columnas:

ID	Categoría	Modelo	Batería	Disponible

Durante la última semana de operaciones se detectaron los siguientes problemas:

- **Problema 1 (Conteo incorrecto de batería crítica)** "El sistema me dice que hay 0 drones con batería crítica, pero yo cuento varios a simple vista en el archivo."
- **Problema 2 (Drones sin batería asignados a repartos)** "Un dron con 5% de batería fue asignado a un reparto largo y se quedó sin energía a mitad de camino. El sistema debería haberlo bloqueado."

Se adjunta la versión actual del script `don_drones_manager.sh` y el archivo `flota.csv`**.**

a) Informe

Investigar el código provisto y realizar un diagnóstico, explicando técnicamente por qué ocurren los problemas reportados. Se debe identificar la línea exacta del fallo y explicar qué hizo el intérprete de Bash en cada caso.

A su vez, se pide entregar un script `don_drones_manager_fix.sh`, en el cual se realicen solamente las modificaciones necesarias para solucionar los problemas 1 y 2.

b) Extensión del Sistema

Don Drones quedó conforme con su informe, pero pide transformar el script en un sistema completo de gestión de flota, **reutilizando y extendiendo la lógica ya corregida en la parte a**.

Requisitos

1. Menú

- El sistema debe presentar un menú con las distintas opciones disponibles, a modo de facilitar su uso.
- Existen dos menúes:
    - Menú de sesión (Requisito 2)
    - Menú principal (Requisitos 3, 4, 5, 6, 7)

2. Usuarios

- Sign Up: usuario y contraseña (no puede ser en blanco). Los usuarios no pueden repetirse.
- Log In: se solicita usuario y contraseña.
    - Una vez autenticado, se redirige al menú principal.
- Log Out.
- Debe existir por defecto un usuario `admin` con contraseña `admin`.

3. Ingresar dron: alta de un nuevo dron con

- **ID:** autogenerado, compuesto por las tres primeras letras de la categoría en mayúscula + número correlativo (ej. `CAR-001` para el primer Cargo).
- **Categoría:** una de las cinco listadas al inicio.
- **Modelo:** nombre del modelo del dron.
- **Batería:** porcentaje entre 0 y 100.
- **Disponible:** Si/No.
- Ejemplo de salida: `CAR-001 - Cargo - SkyHauler X2 - 98% - Disponible`
- Debe quedar guardado en `flota.csv`

4. Asignar dron a reparto: se solicita el **ID** de un dron, y luego se reutiliza la lógica de la **parte a**.

- Adicionalmente, ahora debe verificar que el dron exista.

5. Actualizar dron: se solicita el **ID** de un dron

- Se debe verificar que el dron exista.
- Se consulta su batería actual al usuario y se actualiza.
- Se consulta su disponibilidad al usuario y se actualiza.

6. Reporte de estado de flota

- Indicar cuántos drones tienen batería crítica (menor a 20%), reutilizando la lógica de la **parte a**.
- Listar los drones que tienen batería crítica.
- Ejemplo de salida:

```
Drones con batería crítica: 2
CAR-001 - Cargo - SkyHauler X2 - 8% - Disponible
HEA-001 - Heavy - TitanLift 20 - 19% - Disponible
```

7. Salir: salir del sistema.

Se pide:

- Entregar un script en Bash, con extensión .sh.
- No se acepta el uso de otro lenguaje de programación para implementar la lógica (solo comandos y sintaxis de Bash).
- El sistema debe reutilizar la lógica ya corregida en la parte 1.
- Se debe entregar documentación con los conceptos de implementación relevantes y un ejemplo de uso completo del sistema.

Parte 2 - C

Centro de distribución Springfield



En el centro de distribución de Springfield, cada día ingresan cientos de paquetes que deben pasar por distintas etapas antes de salir a reparto. Todo comienza en el sector de recepción, donde los paquetes son dados de alta en el sistema. Luego pasan al área de escaneo, donde en base a su peso, se calculan ciertos datos de envío, como el costo de envío, y luego son clasificados según su prioridad (*alta* o *normal*), separándolos en colas distintas para su despacho final.

Pero detrás de este flujo aparentemente simple, existe un problema delicado de sincronización: varias personas trabajan al mismo tiempo sobre distintas etapas del sistema, compartiendo áreas de almacenamiento intermedios y colas de despacho. Si la coordinación no es correcta, podrían perderse paquetes, procesarse dos veces o violarse las prioridades de envío. Además, aunque los paquetes prioritarios deben salir antes, tampoco puede permitirse que los paquetes normales queden relegados indefinidamente.

El problema consiste en diseñar un mecanismo que organice este flujo concurrente, asegurando que el procesamiento sea correcto, ordenado y consistente.

Se pide diseñar una solución en C que utilice semáforos POSIX considerando las siguientes restricciones y consideraciones:

Interacciones: El problema debe considerar la interacción de:

- 4 receptores
- 4 escáner
- 4 clasificadores
- 3 despachadores

Paquetes:

- El sistema debe procesar exactamente 300 paquetes.
- Cada paquete debe tener:
    - un identificador único autoincremental
    - un peso aleatorio no mayor a 20 ni menor a 1

Receptores:

- Cada receptor demora entre 1 a 2 segundos en generar un paquete.
- Los receptores insertan los paquetes en un área de almacenamiento “Zona 1” de capacidad máxima 15.
- El programa deberá imprimir en pantalla: “`Receptor X generó paquete Y con peso Z`”

Escáneres:

- Los escáneres extraen paquetes desde la “Zona 1”.
- Cada escáner demora un tiempo random no mayor a 3 segundos.
- Luego del escaneo:
    - si el peso es mayor a 10, el paquete será de prioridad alta
    - en caso contrario, será de prioridad normal
- Además, el costo del envío se calcula como: peso * 100
- Los escáneres insertan luego los paquetes en un área de almacenamiento “Zona 2” de capacidad máxima 15.
- El programa deberá imprimir en pantalla: “`Escáner X procesó paquete Y`”

Clasificadores:

- Los clasificadores extraen paquetes desde el “Zona 2”.
- Cada clasificador demora un tiempo random no mayor a 2 segundos.

- Si el paquete es de prioridad alta, debe enviarse a la "cola de prioridad", de capacidad máxima 10.
- Si el paquete es de prioridad normal, debe enviarse a la "cola normal", de capacidad máxima 15.
- El programa deberá imprimir en pantalla: "`Clasificador X envió paquete Y a cola PRIORIDAD`" o "`Clasificador X envió paquete Y a cola NORMAL`"

Despachador

- Los 3 despachadores toman paquetes desde las dos colas finales.
- Cada despachador demora un tiempo random no mayor a 2 segundos en despachar un paquete.
- Los paquetes de prioridad alta deben tener preferencia, sin embargo, para evitar la postergación excesiva de los paquetes normales, no podrán despacharse más de 3 paquetes prioritarios consecutivos. Si existe al menos un paquete esperando en la cola normal, el siguiente paquete despachado deberá ser normal.
- El programa deberá imprimir en pantalla: "`Despachador X despachó paquete Y - peso Z - prioridad P - costo C`"

Se debe garantizar que:

- No haya más paquetes que la capacidad permitida en cada buffer o cola
- No se pierda ningún paquete
- Ningún paquete sea procesado más de una vez por la misma etapa
- Ningún paquete sea despachado sin haber pasado por recepción, escaneo y clasificación
- Se respete el orden FIFO dentro de cada buffer o cola
- El programa termine únicamente cuando los 300 paquetes hayan sido despachados.

Parte 3. ADA.



Ejercicio: Torneo de Pádel

Se desea desarrollar en ADA una simulación de una jornada de partidos de un torneo de pádel.

El torneo cuenta con **16 parejas**, organizadas en **8 partidos**, y dispone únicamente de **3 canchas**. Debido a que la cantidad de canchas es limitada, no todos los partidos pueden disputarse al mismo tiempo.

Existirá un **Coordinador del torneo**, encargado de administrar los partidos pendientes y asignarlos a las canchas que se encuentren disponibles.

Cada cancha funcionará de manera independiente. Cuando una cancha esté libre deberá solicitar un partido al Coordinador. Si existen partidos pendientes, el Coordinador le asignará uno para disputar.

Cada partido tendrá una duración aleatoria no mayor a **4 segundos**. Una vez finalizado, se determinará aleatoriamente cuál de las dos parejas resultó ganadora.

La cancha deberá informar el resultado al Coordinador y luego quedará nuevamente disponible para solicitar otro partido.

El programa finalizará cuando se hayan disputado los **8 partidos**.

Consideraciones

- Existen **8 partidos definidos desde el comienzo**.
- Existen **3 canchas disponibles**.
- Cada cancha deberá implementarse como una **task de ADA**.
- Deberá existir una **task Coordinador** encargada de administrar los partidos pendientes.
- Las canchas deberán solicitar un partido al Coordinador mediante una cita (rendezvous).
- Al finalizar un partido, la cancha deberá informar el resultado al Coordinador mediante una cita.
- Una cancha solamente podrá disputar **un partido a la vez**.
- Un mismo partido no podrá ser asignado a más de una cancha.
- Cada partido tendrá una duración aleatoria no mayor a **4 segundos**.
- El ganador de cada partido podrá determinarse aleatoriamente.
- Cuando no queden partidos pendientes, las canchas deberán finalizar su ejecución.
- El programa deberá finalizar cuando los **8 partidos hayan terminado**.
- Durante la ejecución se deberán mostrar en pantalla las asignaciones y resultados de los partidos.

Ejemplo de salida

Cancha 1 solicita un partido.

Coordinador:
Partido 1 asignado a Cancha 1.
Pareja 1 vs Pareja 2.

Cancha 2 solicita un partido.

Coordinador:
Partido 2 asignado a Cancha 2.
Pareja 3 vs Pareja 4.

Cancha 1 está disputando el partido...

Cancha 1 finalizó el partido.
Ganador: Pareja 2.

Coordinador recibió el resultado de Cancha 1.

Cancha 1 solicita un nuevo partido.

Todos los partidos han finalizado.

Torneo terminado.

Nota

Se deberán utilizar **tasks y citas de ADA** como fueron vistas en clase. Se valorará que la solución mantenga una estructura clara y simple y que el alumno pueda explicar el funcionamiento de la sincronización implementada.

Parte 4 - Virtualización

Sistema de gestión de acceso a un atracadero



Un pequeño puerto dispone de un único atracadero para recibir embarcaciones.

Diferentes barcos se encuentran navegando en la zona y, en distintos momentos, intentan ingresar al puerto para utilizar el atracadero. Como existe una única posición disponible, nunca puede haber más de un barco utilizando el atracadero al mismo tiempo.

Se deberá desarrollar en Ada una solución basada en microservicios que simule esta situación.

Funcionamiento general

Cada barco deberá implementarse como un microservicio independiente.

Deberán existir al menos tres microservicios de barcos ejecutándose de forma concurrente.

Cada barco repetirá continuamente el siguiente comportamiento:

1. Permanecer navegando durante un tiempo aleatorio.
2. Solicitar acceso al atracadero.
3. Esperar hasta recibir autorización para ingresar.
4. Utilizar el atracadero durante un tiempo aleatorio.
5. Informar que abandona el atracadero.
6. Volver a navegar.
7. Luego de otro tiempo aleatorio, volver a intentar ingresar.

- Solo un barco puede utilizar el atracadero en un momento determinado.
- Si el atracadero está ocupado, los demás barcos que soliciten acceso deberán esperar.
- Cuando el barco que ocupa el atracadero lo libera, uno de los barcos que se encuentre esperando podrá recibir autorización para ingresar.
- Dos barcos nunca podrán recibir simultáneamente autorización para utilizar el atracadero.

La solución deberá contemplar la posibilidad de que varios barcos soliciten acceso prácticamente al mismo tiempo.

Barco 2 vuelve a navegar...

En todo momento deberá cumplirse la siguiente condición:

Cantidad de barcos utilizando el atracadero ≤ 1

Consideraciones

Los distintos microservicios deberán comunicarse entre sí para solicitar acceso, recibir autorización y notificar la liberación del atracadero.

No se establece cómo deberá resolverse dicha comunicación ni qué infraestructura deberá utilizarse. El equipo deberá investigar y seleccionar una solución adecuada.

Entrega

Se deberá entregar:

Facultad de Ingeniería
Bernard Wand-Polak
Cuareim 1451
11.100 Montevideo, Uruguay
Tel. 2902 15 05 Fax 2908 13 70
www.ort.edu.uy

- Código fuente de los diferentes microservicios.
- Instrucciones para ejecutar la solución completa.
- Una explicación de cómo se resolvió la comunicación entre los microservicios.
- Evidencia de ejecución donde pueda observarse que varios barcos compiten por el acceso y que nunca más de uno utiliza el atracadero simultáneamente.

Facultad de Ingeniería
Bernard Wand-Polak
Cuareim 1451
11.100 Montevideo, Uruguay
Tel. 2902 15 05 Fax 2908 13 70
www.ort.edu.uy

Rúbrica de evaluación

Puntaje total: 35 puntos

La evaluación estará compuesta por dos instancias:

Entrega y funcionamiento de los ejercicios: 15 puntos.
Defensa presencial, escrita e individual: 20 puntos.

1. Entrega de los ejercicios – 15 puntos

Para evaluar la entrega se utilizará como referencia la siguiente distribución relativa:

Ejercicio Bash – 5 pts
El alumno es capaz de describir el funcionamiento de la totalidad de los comandos utilizados y sus parámetros. (4 pts)
El programa realiza lo solicitado en su totalidad. (1 pt)
Ejercicio POSIX – 7 pts
El alumno es capaz de describir el funcionamiento de la totalidad de los comandos utilizados y sus parámetros. (5 pts)
El programa realiza lo solicitado en su totalidad. (2 pts)
Ejercicio Ada – 16 pts
El alumno conoce el funcionamiento de cada sección del código y puede explicar qué funcionalidad realiza cada sentencia. En caso de presentar sentencias, bibliotecas, estructuras o soluciones no dadas en clase o ajenas a los laboratorios, deberá demostrar total solvencia en su utilización. (10 pts)
El ejercicio es resuelto utilizando conceptos y analogías con problemas vistos en clase, procurando mantener una solución simple y acorde a los contenidos trabajados. (2 pts)
El programa realiza lo solicitado en su totalidad. (4 pts)
Ejercicio Docker – 7 pts
El alumno es capaz de describir el funcionamiento de la totalidad de los comandos, configuraciones y parámetros utilizados. (4 pts)
El programa y la infraestructura implementada realizan lo solicitado en su totalidad. (3 pts)

El puntaje obtenido en esta sección se ponderará para representar un máximo de 15 puntos de los 35 puntos totales.

2. Defensa presencial, escrita e individual – 20 puntos

Todos los estudiantes deberán realizar una defensa presencial, escrita e individual, cuya fecha y horario serán coordinados posteriormente con el docente.

Durante la defensa se podrán plantear preguntas relacionadas con cualquiera de los ejercicios entregados. El objetivo será comprobar que el estudiante comprende las soluciones presentadas y los conceptos involucrados.

La defensa podrá incluir, entre otros aspectos:

- Explicación del funcionamiento de fragmentos de código de la entrega.
- Explicación de comandos, parámetros, bibliotecas o herramientas utilizadas.
- Análisis de situaciones similares a las resueltas en el obligatorio.
- Identificación de errores o comportamientos en fragmentos de código.
- Propuestas de pequeñas modificaciones sobre la solución entregada.
- Preguntas conceptuales relacionadas con Bash, POSIX, Ada, concurrencia, comunicación entre procesos, Docker y los contenidos utilizados en la resolución.

El estudiante deberá demostrar conocimiento y comprensión de todo el código y las configuraciones que formen parte de su entrega, independientemente de quién haya desarrollado originalmente una determinada sección en caso de trabajos realizados en grupo.

La defensa será evaluada de manera individual, por lo que los integrantes de un mismo grupo podrán obtener calificaciones diferentes.

Consideraciones generales

En caso de realizar el obligatorio con estudiantes de otros grupos, deberán concurrir a la defensa todos juntos.

Esta rúbrica representa una guía para la distribución de los puntos. El docente podrá determinar, según su criterio académico, el puntaje correspondiente ante incumplimientos parciales de alguno de los aspectos indicados.

Errores, situaciones o casuísticas no expresamente contempladas en la rúbrica podrán ser consideradas por el docente durante la evaluación, incluyendo aspectos como documentación, claridad de la solución, organización del código y cumplimiento de las condiciones de entrega.

La defensa presencial, escrita e individual es obligatoria. La no realización de la defensa implicará no acceder al puntaje correspondiente a dicha instancia.

Existe un foro destinado a comunicar aclaraciones, criterios adicionales o decisiones respecto de situaciones particulares que puedan surgir durante el desarrollo del obligatorio.

UNIVERSIDAD ORT
Uruguay

Facultad de Ingeniería
Bernard Wand-Polak
Cuareim 1451
11.100 Montevideo, Uruguay
Tel. 2902 15 05 Fax 2908 13 70
www.ort.edu.uy

RECORDATORIO: IMPORTANTE PARA LA ENTREGA

La entrega de los obligatorios será en formato digital online, a excepción de algunas materias que se entregarán en Bedelía y en ese caso recibirá información específica en el dictado de la misma.

Los principales aspectos a destacar sobre la **entrega online de obligatorios** son:

1. Ingresá al sistema de Gestión.
2. En el menú, seleccioná el ítem “Evaluaciones” y la instancia de evaluación correspondiente, que figura bajo el título “Inscripto”.
3. Para iniciar la entrega hacé clic en el ícono: 
4. Ingresá el número de estudiante de cada uno de los integrantes y hacé clic en “Agregar”. El sistema confirmará que los integrantes estén inscriptos al obligatorio y, de ser así, mostrará el nombre y la fotografía de cada uno de ellos. Una vez agregados todos los integrantes, hacé clic en “Crear equipo”.

   Cualquier integrante podrá:

   • Modificar la integración del equipo.

   • Subir el archivo de la entrega.
5. Seleccioná el archivo que deseás entregar. Verificá el nombre del archivo que aparecerá en la pantalla y hacé clic en “Subir” para iniciar la entrega. Cada equipo (hasta 2 estudiantes) debe entregar **un único archivo en formato zip o rar** (los documentos de texto deben ser pdf, y deben ir dentro del zip o rar). El archivo a subir debe tener **un tamaño máximo de 40mb**

   Cuando el archivo quede subido, se mostrará el nombre generado por el sistema (1), el tamaño y la fecha en que fue subido.
6. El sistema enviará un e-mail a todos los integrantes del equipo informando los detalles del archivo entregado y confirmando que la entrega fue realizada correctamente.
7. Podés cerrar la pestaña de entrega y continuar utilizando Gestión o salir del sistema.
8. La **hora tope para subir el archivo será las 21:00** del día fijado para la entrega.
9. La entrega se podrá realizar desde cualquier lugar (ej. hogar del estudiante, laboratorios de la Universidad, etc).
10. Aquellos de ustedes que presenten alguna dificultad con su inscripción o tengan inconvenientes técnicos, por favor contactarse con la Coordinadora de Cursos o Coordinación Adjunta **antes de las 20:00h** del día de la entrega, a través del mail: adjuntos_ei@ort.edu.uy.